

DEVIXO SOLUTIONS
AI / ML INTERNSHIP PROGRAM

TASK 03

End-to-End Machine Learning Project

Customer Churn Prediction

— Data Analysis & Machine Learning Report —

Student Name: Ammara Fiayyaz

Internship Domain: AI / ML

Submission Date: 06 September 2026

1. Introduction

For this task, I built a complete machine learning project from the ground up, covering everything from raw data to a saved, reusable model. The goal was to work through a real-world problem the way it's actually done in practice: clean the data, engineer useful features, try out several models, tune the best one, and evaluate it properly before calling it done.

I chose to work on customer churn prediction essentially, figuring out whether a bank customer is likely to close their account based on things like their age, balance, credit score, how many products they use, and whether they're an active member. It's a common problem in the banking and telecom industry, and a good one for practicing the full ML workflow.

The sections below walk through what I did at each stage, what I found, and where the model still falls short.

2. Dataset

I used the Churn_Modelling.csv dataset, which has 10,000 customer records across 14 columns. That's well above the 3,000-row minimum for this task.

Original Dataset Columns

- RowNumber
- CustomerId
- Surname
- CreditScore
- Geography
- Gender
- Age
- Tenure
- Balance
- NumOfProducts
- HasCrCard
- IsActiveMember
- EstimatedSalary
- Exited

The column I'm trying to predict is Exited:

- 0 — the customer stayed
- 1 — the customer left

First Look at the Data

- 10,000 rows, 14 columns
- No missing values
- No duplicate rows

Since the data came in clean, I didn't need to spend time on imputation or de-duplication — I could move straight into preparing the features.

3. Data Preparation

3.1 Data Cleaning

Three columns weren't going to help the model in any meaningful way, so I dropped them right away:

- RowNumber
- CustomerId
- Surname

These are just identifiers — they don't say anything about a customer's behavior. After dropping them, I was left with 11 columns:

- CreditScore
- Geography
- Gender
- Age
- Tenure
- Balance
- NumOfProducts
- HasCrCard
- IsActiveMember
- EstimatedSalary
- Exited

3.2 Feature Engineering

I added two new features that I thought could give the models a bit more signal to work with.

Balance-to-Salary Ratio

This compares how much money a customer keeps in the bank relative to what they earn:

$$\text{BalanceSalaryRatio} = \text{Balance} / (\text{EstimatedSalary} + 1)$$

I added 1 to the salary just to avoid a divide-by-zero error for anyone with a salary of 0.

Age Group

I also bucketed customer age into groups, since churn behavior often varies by life stage more than by exact age:

- Young: 0–30
- Adult: 30–40
- Middle_Age: 40–50
- Senior: 50–100

This gave me a new categorical column, AgeGroup.

3.3 Categorical Encoding

Geography, Gender, and AgeGroup are all categorical, so I one-hot encoded them to turn them into numeric columns the models could actually use:

- Geography_Germany
- Geography_Spain
- Gender_Male
- AgeGroup_Adult
- AgeGroup_Middle_Age
- AgeGroup_Senior

I used `drop_first=True` to avoid redundant columns. After encoding, the dataset had 16 columns in total, including the target.

3.4 Feature Selection and Analysis

Once the target column was separated out, I was left with 15 input features. Rather than dropping anything at this stage, I ran a quick Random Forest and looked at its feature importances just to get a sense of which variables were actually pulling weight.

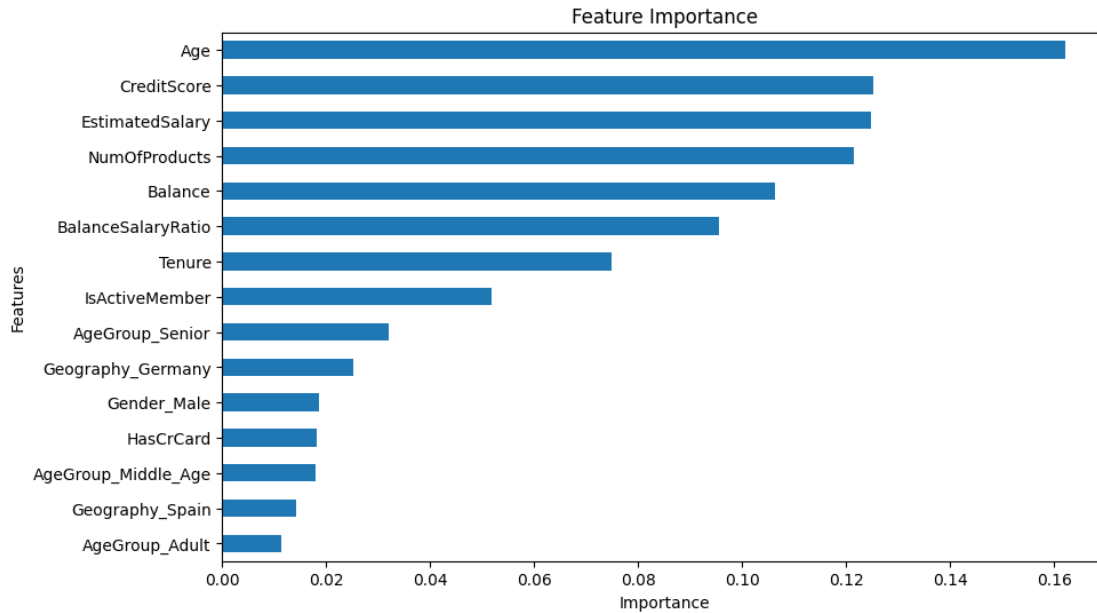


Figure 1 — Initial feature importance from the Random Forest model, used to guide feature analysis before model selection.

Age, CreditScore, and EstimatedSalary came out on top in this early pass. I decided to keep every feature going into model training rather than trimming anything down, since a few of the lower-ranked features still turned out to matter more once I trained the final model.

3.5 Train-Test Split

I split the data 80/20 into training and testing sets, using stratified sampling so the churn/non-churn ratio stayed consistent across both sets.

Dataset	Records	Features
Training Set	8,000	15
Testing Set	2,000	15

3.6 Data Scaling

I scaled the features using StandardScaler, fitting it only on the training set and then applying that same transformation to the test set. This keeps the test data from leaking any information into the preprocessing step.

Scaling mattered most for the SVM model, since it's sensitive to feature magnitude in a way that tree-based models aren't.

4. Machine Learning Models

I trained four different models for this project, which covers the task requirement of at least four:

- Random Forest
- Support Vector Machine (SVM)
- Gradient Boosting
- Decision Tree

Each one was evaluated on accuracy along with a full classification report, so I could compare not just overall accuracy but how well each model actually caught churn cases.

5. Model Selection and Initial Results

5.1 Random Forest

- Accuracy: 86.1%
- Churn Recall: 43%
- Churn F1-score: 55%

This model was solid at spotting customers who stayed, but it missed more than half of the customers who actually churned.

5.2 Support Vector Machine

- Accuracy: 86.1%
- Churn Recall: 43%
- Churn F1-score: 56%

SVM landed at almost the exact same accuracy as Random Forest, with a marginally better F1-score on the churn class.

5.3 Gradient Boosting

- Accuracy: 87.1%
- Churn Recall: 49%
- Churn F1-score: 61%

This was the best of the four right out of the box, both on accuracy and on the churn F1-score, so it became my main candidate going into the optimization stage.

5.4 Decision Tree

- Accuracy: 77.9%
- Churn Recall: 49%

- Churn F1-score: 47%

A single decision tree clearly overfits or underperforms compared to the ensemble methods here — its accuracy dropped noticeably below the other three.

6. Model Comparison

Putting all four side by side made it easy to see where things stood after the first round of training:

Model	Accuracy	Churn Recall	Churn F1-score
Gradient Boosting	87.1%	49%	61%
Random Forest	86.1%	43%	55%
SVM	86.1%	43%	56%
Decision Tree	77.9%	49%	47%

Gradient Boosting was the clear front-runner at this point, but I didn't want to lock it in without checking how consistent it was — that's what the cross-validation step was for.

7. Cross-Validation

I ran 5-fold cross-validation on all four models to see whether the initial results held up or were just a lucky train/test split.

Model	Cross-Validation Accuracy
Random Forest	85.84%
SVM	85.49%
Gradient Boosting	86.39%
Decision Tree	79.20%

Gradient Boosting still came out on top, and its cross-validation accuracy (86.39%) was close to its test accuracy (87.1%), which told me the model wasn't just overfitting to the test split.

8. Hyperparameter Tuning

Next, I ran GridSearchCV with 5-fold cross-validation to try to squeeze more performance out of Gradient Boosting. I searched over:

- n_estimators: 100, 150
- learning_rate: 0.05, 0.1
- max_depth: 2, 3

That's 8 combinations in total.

Best Parameters

Parameter	Best Value
Learning Rate	0.1
Maximum Depth	3
Number of Estimators	100

Best Cross-Validation Accuracy: 86.39%

Interestingly, the grid search landed on the exact same configuration I'd already been using by default. So in this case, tuning confirmed the model was already well set up rather than finding a better setup — a reminder that scikit-learn's defaults for Gradient Boosting are often a reasonable starting point.

9. Final Model Evaluation

With the tuned model locked in, I ran it against the held-out test set for the final evaluation.

Final Accuracy: 87.1%

Classification Report

Class	Precision	Recall	F1-score	Support
0 – Did Not Churn	0.88	0.97	0.92	1593
1 – Churned	0.80	0.49	0.61	407
Overall Accuracy			0.87	2000
Macro Average	0.84	0.73	0.77	2000
Weighted Average	0.86	0.87	0.86	2000

The 87.1% overall accuracy looks good on paper, but the churn class tells the more honest story: 80% precision but only 49% recall. In plain terms, when the model flags someone as likely to

churn, it's usually right — but it still misses about half of the customers who actually do churn. For a churn-prediction use case, that's the number I'd want to improve first.

10. Feature Importance Analysis

Once the final model was trained, I pulled its feature importances to see what it was actually relying on to make predictions.

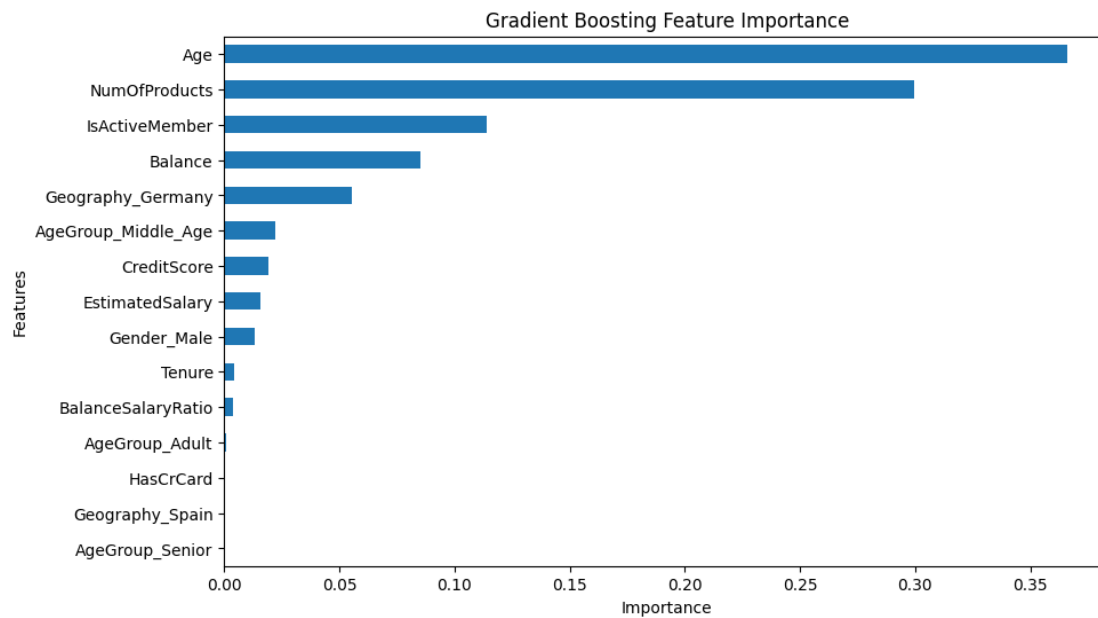


Figure 2 — Feature importance from the final tuned Gradient Boosting model.

Rank	Feature	Importance
1	Age	0.366369
2	NumOfProducts	0.299800
3	IsActiveMember	0.113926
4	Balance	0.085146
5	Geography_Germany	0.055617
6	AgeGroup_Middle_Age	0.021963
7	CreditScore	0.019006
8	EstimatedSalary	0.015478
9	Gender_Male	0.013155

Rank	Feature	Importance
10	Tenure	0.004154
11	BalanceSalaryRatio	0.003734
12	AgeGroup_Adult	0.000823
13	HasCrCard	0.000518
14	Geography_Spain	0.000308
15	AgeGroup_Senior	0.000002

Age, NumOfProducts, and IsActiveMember dominate the model's decisions, together accounting for roughly 78% of the total importance. Everything else — credit score, salary, tenure, geography — plays a much smaller supporting role. It's worth noting how different this is from the earlier Random Forest pass in Section 3.4, where CreditScore and EstimatedSalary ranked much higher. That shift shows how much a model's own internal logic can reshape what 'important' even means for a given algorithm.

11. Model Saving and Reusability

To make the model reusable without retraining, I saved both the trained model and the fitted scaler using Joblib:

- `best_gradient_boosting_model.pkl`
- `scaler.pkl`

I then loaded both files back in to confirm everything worked end-to-end, and got:

Model loaded successfully!

Scaler loaded successfully!

That confirms the pipeline is genuinely reusable — someone could load these two files and start making predictions on new customer data without having to retrain anything.

12. Complete Machine Learning Workflow

Here's the full sequence I followed from start to finish:

1. Dataset Collection
2. Data Loading and Inspection
3. Missing Value and Duplicate Checking

4. Data Cleaning
5. Feature Engineering
6. Categorical Encoding
7. Feature Selection and Importance Analysis
8. Train-Test Split
9. Feature Scaling
10. Model Training
11. Model Comparison
12. Cross-Validation
13. Hyperparameter Tuning
14. Final Model Evaluation
15. Feature Importance Analysis
16. Model Saving
17. Model Loading Verification

13. Final Model Selection

Taking everything together, I went with Gradient Boosting as the final model. Here's a summary of where it landed:

- 87.1% Test Accuracy
- 86.39% Cross-Validation Accuracy
- 80% Churn Precision
- 49% Churn Recall
- 61% Churn F1-score

It beat out Random Forest, SVM, and Decision Tree on both accuracy and churn F1-score, and it stayed consistent across cross-validation, so it was the natural choice to save as the final model.

14. Limitations

The model does well overall, but I ran into a few things worth being upfront about.

14.1 Churn Recall

A recall of 49% on the churn class means the model still misses about half the customers who actually leave. If this were going into production for something like a retention campaign, that miss rate would need to come down.

14.2 Class Imbalance

The test set breaks down as:

- 1,593 customers who didn't churn
- 407 customers who did

With roughly 4x more non-churn examples, the model naturally leans toward predicting the majority class more confidently, which is a big part of why churn recall lags behind overall accuracy.

14.3 Limited Hyperparameter Search

I only tested 8 combinations of parameters. A wider search — more values for learning rate, tree depth, or subsampling — could potentially find a configuration that does better on churn recall specifically, rather than just overall accuracy.

14.4 Feature Limitations

The dataset only captures a fixed snapshot of customer attributes. It doesn't include things like transaction history, support interactions, or usage patterns over time — data that could plausibly do a lot to improve churn detection.

15. Future Improvements

If I were to keep working on this, here's what I'd try next:

1. Run a wider hyperparameter search with more parameter combinations.
2. Try XGBoost as an additional boosting model to compare against Gradient Boosting.
3. Address the class imbalance directly, using class weighting or resampling techniques like SMOTE.
4. Optimize specifically for churn recall rather than overall accuracy, since that's the metric that matters most for this use case.
5. Bring in more behavioral data — transaction history, support tickets, usage frequency — if it becomes available.
6. Test a few more algorithms beyond the four used here.
7. Wrap the model in a simple web interface so it's actually usable by someone outside the notebook.
8. Build a Streamlit app where a user can enter customer details and get a churn prediction back.
9. Once deployed, monitor the model's performance over time and retrain it as new data comes in.

16. Conclusion

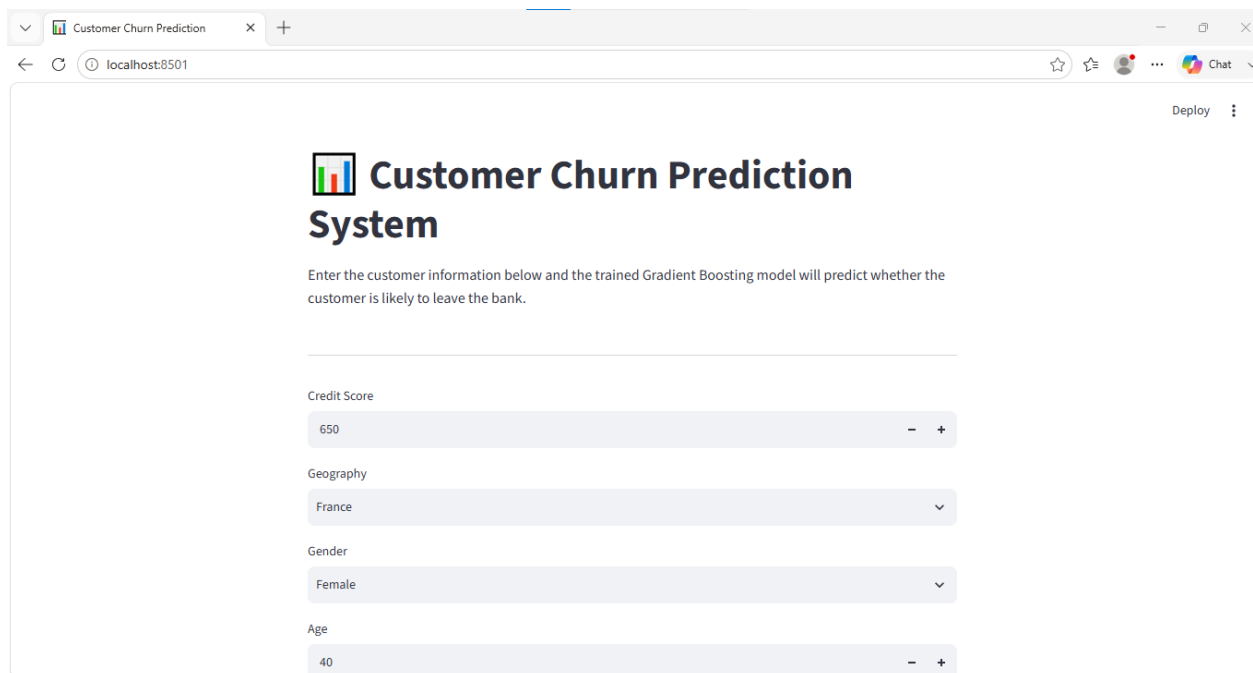
This project took me through the full machine learning workflow, start to finish — cleaning and preparing the data, engineering new features, encoding categories, training and comparing four different models, validating and tuning the best one, digging into what it was actually learning, and saving it in a form that can be reused.

Out of Random Forest, SVM, Gradient Boosting, and Decision Tree, Gradient Boosting came out ahead, with 87.1% test accuracy and 86.39% cross-validation accuracy. I saved the model and its scaler with Joblib and confirmed both reload correctly.

It's not a perfect model — churn recall is the clear area to improve — but it's a solid, working example of how a churn-prediction system comes together end to end, and it gives me a good base to build on if I revisit this later.

17. Streamlit Prediction Interface

The Streamlit application provides a simple user interface for entering customer information and predicting whether the customer is likely to churn. The application uses the saved Gradient Boosting model developed during the machine learning process.



The screenshot shows a web browser window with a single tab titled "Customer Churn Prediction". The address bar shows "localhost:8501". The page features a title "Customer Churn Prediction System" with a small bar chart icon. Below the title is a descriptive text: "Enter the customer information below and the trained Gradient Boosting model will predict whether the customer is likely to leave the bank." The form contains four input fields: "Credit Score" with a numeric value of 650, "Geography" with a dropdown menu set to "France", "Gender" with a dropdown menu set to "Female", and "Age" with a numeric value of 40. Each field has a minus and a plus button for adjustment. A "Deploy" button is visible in the top right corner of the application area.

Customer Churn Prediction

localhost:8501

Deploy

Yes

Is Active Member

Yes

Estimated Salary

50000.00

Predict Churn

✓

The customer is likely to stay.

Developed as part of the Devixo Solutions AI/ML Internship Project.